

Nama : Achmad Pradita Dwi Firmansyah
NIM : 254107020130
Kelas / Absen : TI – 1G / 01

JOBSHEET V

BRUTE FORCE DAN DIVIDE CONQUER

5.1.1. Kode Program

Class Faktorial :

```
package BruteForceDivideConquer.minggu5;
public class Faktorial {
    int faktorialBF(int n) {
        int fakto = 1;
        for (int i = 1; i <= n; i++) {
            fakto = fakto * i;
        }
        return fakto;
    }
    int faktorialDC(int n) {
        if (n == 1) {
            return 1;
        } else {
            int fakto = n * faktorialDC(n-1);
            return fakto;
        }
    }
}
```

Class MainFaktorial :

```
package BruteForceDivideConquer.minggu5;
import java.util.Scanner;
public class MainFaktorial {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan nilai: ");
        int nilai = input.nextInt();

        Faktorial fk = new Faktorial();
        System.out.println("Nilai faktorial "+nilai+ " menggunakan BF:
"+fk.faktorialBF(nilai));
        System.out.println("Nilai faktorial "+nilai+ " menggunakan DC:
"+fk.faktorialDC(nilai));
    }
}
```

5.1.2. Verifikasi Hasil Percobaan

Cocokkan hasil compile kode program anda dengan gambar berikut ini.

```
Masukkan nilai: 5
Nilai faktorial 5 menggunakan BF: 120
Nilai faktorial 5 menggunakan DC: 120
```

```
Masukkan nilai: 5
Nilai faktorial 5 menggunakan BF: 120
Nilai faktorial 5 menggunakan DC: 120
PS C:\PENYIMPANAN\Documents\G\PraktikumAlgoritmaDanStrukturData>
```

5.1.3. Pertanyaan

1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!

Pada bagian kode didalam if, program akan berhenti jika nilai n adalah 1.

Pada bagian kode didalam else, program akan mencari nilai fakto dengan menghitung nilai n dikalikan nilai n yang terus berkurang menjadi kecil.

2. Apakah memungkinkan perulangan pada method faktorialBF() diubah selain menggunakan for? Buktikan!

Perulangan pada method faktorialBF() bisa diubah menggunakan while dan do while

```
Masukkan nilai: 5
Nilai faktorial 5 menggunakan BF: 120
Nilai faktorial 5 menggunakan DC: 120
Nilai faktorial 5 menggunakan BF dan perulangan While: 120
Nilai faktorial 5 menggunakan DC dan perulangan Do While: 120
PS C:\PENYIMPANAN\Documents\G\PraktikumAlgoritmaDanStrukturData>
```

Kode Program :

```
int faktorialBFWhile (int n) {
    int fakto = 1;
    int i = 1;
    while (i <= n) {
        fakto = fakto * i;
        i++;
    }
    return fakto;
}

int faktorialBFDoWhile (int n) {
    int fakto = 1;
    int i = 1;
    do {
        fakto = fakto * i;
        i++;
    } while (i <= n);
    return fakto;
}
```

```
System.out.println("Nilai faktorial "+nilai+ " menggunakan BF dan perulangan While: "+fk.faktorialBFWhile(nilai));
System.out.println("Nilai faktorial "+nilai+ " menggunakan DC dan perulangan Do While: "+fk.faktorialBFDoWhile(nilai));
```

3. Jelaskan perbedaan antara fakto *= i; dan int fakto = n * faktorialDC(n-1);!

Pada fakto *= i, sudah di inisialisasi nilai dari fakto yaitu 1, dan nilai fakto tersebut di kalikan dengan nilai i dan hasilnya dimasukkan menjadi nilai fakto baru, dan perkalian tersebut dilakukan sebanyak nilai n.

Pada fakto = n * faktorialDC(n-1) nilai fakto belum di inisiasi sehingga nilai fakto didapat dari nilai n dikali dengan nilai (n-1) yang memanggil fungsi tersebut sampai nilai n akan berkurang hingga n = 1.

4. Buat Kesimpulan tentang perbedaan cara kerja method faktorialBF() dan faktorialDC()!

Pada method faktorialBF() program menghitung nilai fakto dari awal hingga nilai n. Pada method faktorialDC() program menghitung nilai fakto dengan membagi nilai n menjadi beberapa bagian dengan fungsi rekursif dan nilai n-1.

5.1.4. Kode Program

Class Pangkat :

```
package BruteForceDivideConquer.minggu5;
public class Pangkat {
    int nilai, pangkat;
    public Pangkat(int n, int p){
        nilai = n;
        pangkat = p;
    }
    int pangkatBF(int a, int n) {
        int hasil = 1;
        for (int i = 0; i < n; i++) {
            hasil = hasil*a;
        }
        return hasil;
    }
    int pangkatDC(int a, int n) {
        if (n == 1) {
            return a;
        }else {
            if (n % 2 == 1) {
                return (pangkatDC(a, n/2) * pangkatDC(a, n/2) * a);
            }else {
                return (pangkatDC(a, n/2) * pangkatDC(a, n/2));
            }
        }
    }
}
```

Class MainPangkat :

```
package BruteForceDivideConquer.minggu5;
import java.util.Scanner;
public class MainPangkat {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan jumlah elemen: ");
        int elemen = input.nextInt();
        Pangkat [] png = new Pangkat[elemen];
        for (int i = 0; i < elemen; i++) {
            System.out.print("Masukkan nilai basis elemen ke-"+(i+1)+" : ");
            int basis = input.nextInt();
        }
    }
}
```

```

        System.out.print("Masukkan nilai pangkat elemen ke-"+(i+1)+" : ");
        int pangkat = input.nextInt();
        png [i] = new Pangkat(basis, pangkat);
    }
    System.out.println("HASIL PANGKAT BRUTEFORCE:");
    for (Pangkat p : png) {
        System.out.println(p.nilai+"^"+p.pangkat+" : "+p.pangkatBF(p.nilai, p.pangkat));
    }
    System.out.println("HASIL PANGKAT DIVIDE AND CONQUER:");
    for (Pangkat p : png) {
        System.out.println(p.nilai+"^"+p.pangkat+" : "+p.pangkatDC(p.nilai, p.pangkat));
    }
}
}
}

```

5.1.5. Verifikasi Hasil Percobaan

Pastikan output yang ditampilkan sudah benar seperti di bawah ini.

```

Masukkan jumlah elemen: 3
Masukkan nilai basis elemen ke-1: 2
Masukkan nilai pangkat elemen ke-1: 3
Masukkan nilai basis elemen ke-2: 4
Masukkan nilai pangkat elemen ke-2: 5
Masukkan nilai basis elemen ke-3: 6
Masukkan nilai pangkat elemen ke-3: 7
HASIL PANGKAT BRUTEFORCE:
2^3: 8
4^5: 1024
6^7: 279936
HASIL PANGKAT DIVIDE AND CONQUER:
2^3: 8
4^5: 1024
6^7: 279936

```

```

Masukkan jumlah elemen: 3
Masukkan nilai basis elemen ke-1: 2
Masukkan nilai pangkat elemen ke-1: 3
Masukkan nilai basis elemen ke-2: 4
Masukkan nilai pangkat elemen ke-2: 5
Masukkan nilai basis elemen ke-3: 6
Masukkan nilai pangkat elemen ke-3: 7
HASIL PANGKAT BRUTEFORCE:
2^3: 8
4^5: 1024
6^7: 279936
HASIL PANGKAT DIVIDE AND CONQUER:
2^3: 8
4^5: 1024
6^7: 279936
PS C:\PENYIMPANAN\Documents\G\PraktikumAlgoritmaDanStrukturData>

```

5.1.6. Pertanyaan

1. Jelaskan mengenai perbedaan 2 method yang dibuat yaitu pangkatBF() dan pangkatDC()!

Pada method pangkatBF() program akan mengalikan nilai dengan nilai secara berulang sebanyak jumlah pangkat. Sedangkan pada method pangkatDC() program membagi pangkat menjadi beberapa bagian dengan membagi jumlah pangkat dengan 2.

2. Apakah tahap *combine* sudah termasuk dalam kode tersebut? Tunjukkan!

Tahap *combine* sudah termasuk kedalam kode tersebut, dengan menggabungkan hasil dari pembagian program dengan menggunakan operator perkalian.

```

pangkatDC(a, n/2) * pangkatDC(a, n/2) * a
pangkatDC(a, n/2) * pangkatDC(a, n/2)

```

3. Pada method `pangkatBF()` terdapat parameter untuk melewati nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class `Pangkat` telah ada atribut nilai dan pangkat, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method `pangkatBF()` yang tanpa parameter?

Method tersebut tidak relevan jika memiliki parameter, karena method tersebut bisa dibuat tanpa parameter seperti berikut.

```

int pangkatBF() {
    int hasil = 1;
    for (int i = 0; i < pangkat; i++) {
        hasil = hasil*nilai;
    }
    return hasil;
}

```

4. Tarik tentang cara kerja method `pangkatBF()` dan `pangkatDC()`!
 Method `pangkatBF()` menginisialisasi hasil dengan 1, kemudian melakukan perkalian nilai sebanyak jumlah pangkat, dan dimasukkan ke dalam variable hasil yang akan di kembalikan.
 Method `pangkatDC()` mengecek nilai dari pangkat jika pangkat tidak sama dengan 1, maka dilakukan pengecekan apakah pangkat bilangan ganjil, maka dilakukan 3 pembagian program yaitu nilai, pangkat / 2; nilai, pangkat / 2; nilai, dan dilakukan penggabungan kembali dengan menggunakan operator perkalian. Jika pangkat bilangan genap, maka program dibagi 2 bagian, yaitu nilai, pangkat / 2 dan nilai, pangkat / 2, dan dilakukan penggabungan kembali dengan operator perkalian.

5.1.1. Kode Program

```

Class Sum :
package BruteForceDivideConquer.minggu5;
public class Sum {
    double keuntungan[];
    Sum(int el) {
        keuntungan = new double[el];
    }
    double totalBF() {
        double total = 0;
        for (int i = 0; i < keuntungan.length; i++) {
            total = total + keuntungan[i];
        }
        return total;
    }
    double totalDC(double arr[], int l, int r) {
        if (l == r) {
            return arr[l];
        }
        int mid = (l + r) / 2;
        double lsum = totalDC(arr, l, mid);

```

```

        double rsum = totalDC(arr, mid + 1, r);
        return lsum + rsum;
    }
}

```

Class MainSum :

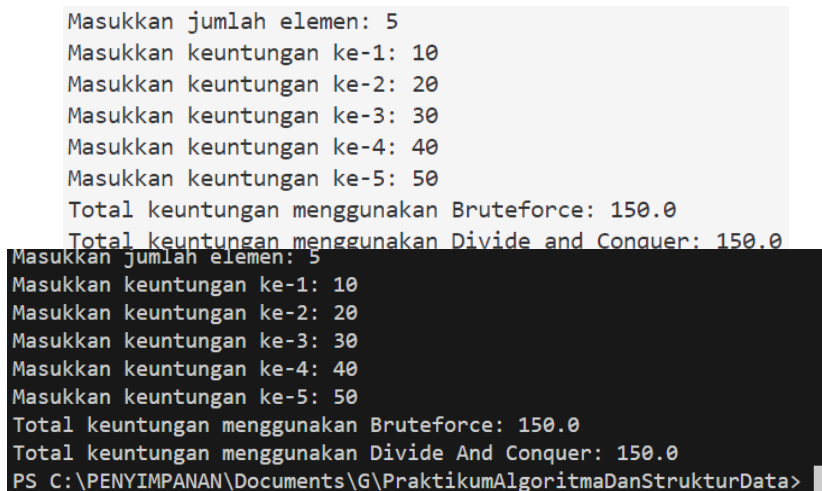
```

package BruteForceDivideConquer.minggu5;
import java.util.Scanner;
public class MainSum {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan jumlah elemen: ");
        int elemen = input.nextInt();
        Sum sm = new Sum(elemen);
        for (int i = 0; i < elemen; i++) {
            System.out.print("Masukkan keuntungan ke-" + (i+1) + ": ");
            sm.keuntungan[i] = input.nextDouble();
        }
        System.out.println("Total keuntungan menggunakan Bruteforce: " + sm.totalBF());
        System.out.println("Total keuntungan menggunakan Divide And Conquer: " + sm.totalDC(sm.keuntungan, 0, elemen-1));
    }
}

```

5.1.2. Verifikasi Hasil Percobaan

Cocokkan hasil compile kode program anda dengan gambar berikut ini.



```

Masukkan jumlah elemen: 5
Masukkan keuntungan ke-1: 10
Masukkan keuntungan ke-2: 20
Masukkan keuntungan ke-3: 30
Masukkan keuntungan ke-4: 40
Masukkan keuntungan ke-5: 50
Total keuntungan menggunakan Bruteforce: 150.0
Total keuntungan menggunakan Divide and Conquer: 150.0
Masukkan jumlah elemen: 5
Masukkan keuntungan ke-1: 10
Masukkan keuntungan ke-2: 20
Masukkan keuntungan ke-3: 30
Masukkan keuntungan ke-4: 40
Masukkan keuntungan ke-5: 50
Total keuntungan menggunakan Bruteforce: 150.0
Total keuntungan menggunakan Divide And Conquer: 150.0
PS C:\PENYIMPANAN\Documents\G\PraktikumAlgoritmaDanStrukturData>

```

5.1.3. Pertanyaan

1. Kenapa dibutuhkan variable mid pada method TotalDC()?

Variable mid digunakan untuk membagi nilai array menjadi 2 bagian left dan right, tanpa variable mid, tidak bisa dilakukan pembagian array.

2. Untuk apakah statement di bawah ini dilakukan dalam TotalDC()?

```

double lsum = totalDC(arr, l, mid);
double rsum = totalDC(arr, mid+1, r);

```

lsum digunakan untuk menjumlahkan array di bagian kiri, sedangkan rsum digunakan untuk

menjumlahkan array bagian kanan dengan memanggil kembali fungsi tersebut.

3. Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?

```
return lsum+rsum;
```

Penjumlahan lsum dan rsum diperlukan untuk menggabungkan kembali hasil dari penjumlahan array yang sebelumnya dilakukan pembagian menjadi 2 bagian array, sehingga didapatkan penjumlahan array secara total.

4. Apakah base case dari totalDC()?

Base case dari totalDC() yaitu ketika nilai l dan nilai r sama, yang dimana variable l dan r digunakan sebagai indeks dari array tersebut.

5. Tarik Kesimpulan tentang cara kerja totalDC()

Method totalDC() melakukan pengecekan nilai l dan r, jika nilai keduanya sama, maka nilai yang dihasilkan yaitu nilai dari array pada indeks ke l. Jika nilai keduanya berbeda, maka dilakukan pencarian nilai tengah. Kemudian dilakukan penjumlahan masing masing bagian dan penggabungan kembali masing masing bagian dengan melakukan penjumlahan bagian kiri dan kanan.